Self-Supervised Session-Graph Recommendation with SR-GNN: Midterm Status Report and Robustness Analysis

Anonymous Submission
CSCI-566: Deep Learning and Its Applications
University of Southern California

Abstract

Session-based recommendation (SBR) predicts the next item a user is likely to interact with given a sequence of anonymous, short-term browsing events. Graph Neural Networks (GNNs), exemplified by the Session-based Recommendation with Graph Neural Networks (SR-GNN) architecture, capture complex, non-consecutive item transitions by converting sequential sessions into directed graphs. However, existing session-graph models exhibit severe sensitivity to noisy clicks, random exploratory browsing, and extreme data sparsity. In this midterm report, we present the progress of our CSCI-566 project: (1) we implement a modular, reproducible, and modernized PyTorch implementation of the SR-GNN baseline; (2) we design a deterministic data pipeline with recursive frequency filtering, temporal splitting, and cryptographic artifact verification on the Diginetica and RetailRocket benchmarks; (3) we successfully reproduce the published SR-GNN baseline, achieving 49.55% Recall@20 and 17.11% MRR@20 on Diginetica (matching published numbers within 1.18 points under frozen chronological splits); (4) we conduct a comprehensive noise-robustness benchmark across nine synthetic corruption regimes (deletion, substitution, and mixed perturbations at 5%, 10%, and 20% ratios) under strict target invariance; and (5) we formulate our upcoming self-supervised contrastive learning framework (SSL) designed to regularize session graph representations against accidental noise.

1 Introduction

Modern e-commerce and media platforms increasingly operate under strict user privacy constraints and transient visitor patterns. Unlike traditional recommendation systems that rely on rich historical interaction matrices, user profiles, or matrix factorization (Rendle et al., 2010), *session-based recommendation* (SBR) must operate strictly on anonymous, short-term sessions where the user's permanent identifier is unknown (Hidasi et al., 2016). In a typical session, a user interacts with a few items (often fewer than five clicks) within a narrow temporal window.

Early session-based systems employed Markov chains or Recurrent Neural Networks (RNNs) such as GRU4Rec (Hidasi et al., 2016) and attention-based sequential models such as SASRec (Kang and McAuley, 2018). While effective at modeling rigid sequential transitions, pure sequence models struggle to account for non-consecutive browsing patterns, user backtracking, or cyclic transitions. To overcome these limitations, Wu et al. (2019) proposed SR-GNN, which maps each session into a directed session graph. By employing Gated Graph Neural Networks (GGNN) (Li et al., 2015) with attention-weighted readout pooling, SR-GNN combines local preference (the immediate last click) with global session intent (the collective graph propagation).

Despite their empirical success, graph-based session models face significant practical vulnerabilities:

1. **High Sensitivity to Clickstream Noise:** Real-world user sessions frequently contain accidental clicks, misdirected navigation, and exploratory drift. Because session graphs are small (median 3–4 nodes), even a single extraneous edge can distort the message passing across the entire graph.
2. **Data Sparsity and Overfitting:** Over 70% of sessions contain fewer than five interactions, leading to sparse graphs where node embeddings are prone to severe overfitting during training.
3. **Lack of Reproducibility in Benchmarks:** Many published studies rely on non-deterministic data filtering, random validation splits that cause temporal leakage, or inconsistent tie-breaking rules.

Midterm Contributions. To resolve these challenges and establish a rigorous foundation for our final self-supervised model, our project has achieved the following milestones up to the midterm checkpoint:

- **Modernized PyTorch SR-GNN Implementation:** We developed a clean, vectorized, and modular implementation of SR-GNN in modern PyTorch, fixing legacy flaws (such as non-zero vectors in padding row 0 and unseeded graph batching) while preserving the original mathematical formulation.
- **Deterministic Data Pipeline with Verification:** We built an automated preprocessing pipeline that enforces recursive frequency filtering, strict chronological train/validation/test windowing, contiguous 1-based item indexing, and cryptographic SHA-256 artifact verification.
- **Baseline Reproduction:** We conducted preliminary training and evaluation on the Diginetica benchmark (seed 2028). The baseline reproduces published SR-GNN numbers within 1.18 absolute points on Recall@20 (49.55% vs. 50.73%) and 0.48 points on MRR@20 (17.11% vs. 17.59%) under our strict, non-leaking chronological split.
- **Offline Robustness Benchmark:** We evaluated the trained baseline across nine controlled noisy test sets spanning deletion, substitution, and mixed corruption at 5%, 10%, and 20% ratios, strictly preserving target invariance. We find that substitution noise degrades performance twice as severely as deletion.
- **Self-Supervised Contrastive Formulation:** We completed the mathematical formulation and architectural design for our upcoming session-level InfoNCE contrastive extension, featuring recency-preserving graph augmentation.

2 Related Work

2.1 Sequential Recommendation

Early approaches to session-based recommendation relied on Markov Decision Processes (MDPs) and Factorized Personalized Markov Chains (FPMC) (Rendle et al., 2010), which model item transitions under first-order Markovian assumptions. Hidasi et al. (2016) pioneered deep learning for SBR with GRU4Rec, utilizing gated recurrent units and session-parallel mini-batch training. Subsequent architectures integrated attention mechanisms to highlight informative items within a sequence, including NARM and STAMP. More recently, self-attentive sequential recommendation (SASRec) (Kang and McAuley, 2018) demonstrated that Transformer-style multi-head self-attention outperforms recurrent architectures by capturing long-range dependencies across the entire sequence. However, pure sequential models view sessions strictly as 1D arrays, failing to capture bidirectional transitions and complex graph topologies.

2.2 Graph Neural Networks in Recommendation

Graph Neural Networks have transformed recommender systems by explicitly modeling relational inductive biases (Kipf and Welling, 2017; Veličković et al., 2018; He et al., 2020). In session recommendation, SR-GNN (Wu et al., 2019) modeled item transitions as directed graphs and utilized gated message passing to learn item vector representations. Subsequent models extended this paradigm: GC-SAN incorporated self-attention layers on top of graph embeddings, while GCE-GNN explored global graph contexts across sessions. Nevertheless, these models remain predominantly supervised and rely heavily on cross-entropy next-item prediction, rendering them susceptible to sparse signals and clickstream perturbations.

2.3 Self-Supervised Learning in RecSys

Self-Supervised Learning (SSL) has emerged as an effective paradigm to counter label sparsity by maximizing mutual information between multiple augmented views of the same instance (Oord et al., 2018). In sequential recommendation, S^3-Rec (Zhou et al., 2020) applied mutual information maximization between items, sequences, and contexts. For graph-based recommendation, S^2-DHCN (Xia et al., 2021) integrated hypergraph modeling with contrastive learning, showing substantial improvements in sparse settings. Our project builds upon these insights by developing recency-aware contrastive objectives specifically tailored to SR-GNN graph structures.

3 Problem Formulation and Data Pipeline

3.1 Problem Formulation

Let $V = \{v_1, v_2, \ldots, v_{|V|}\}$ denote the universe of all unique candidate items. An anonymous user session is represented as an ordered sequence of

interacted item IDs:

$$S = [s_1, s_2, \ldots, s_l], \quad s_k \in V \tag{1}$$

where l is the sequence length and s_k is the item clicked at time step k. Given an observed prefix sub-sequence $S_{1:t} = [s_1, s_2, \ldots, s_t]$ (with $1 \leq t < l$), the objective of session-based recommendation is to predict the immediate next item $s_{t+1} \in V$. Specifically, the recommendation model outputs a ranking score vector:

$$\hat{\mathbf{y}} = f(S_{1:t}) \in \mathbb{R}^{|V|} \tag{2}$$

where $\hat{y}_i$ represents the unnormalized preference score for candidate item $v_i \in V$. The top-K items with the highest scores are returned as the recommendation set.

3.2 Dataset Acquisition and Preprocessing

We evaluate our methods on two benchmark e-commerce datasets:

- **Diginetica (CIKM Cup 2016):** Clickstream event logs extracted from an e-commerce platform over a period of five months.
- **RetailRocket:** Multi-behavior e-commerce interaction data (item views, add-to-carts, transactions) collected over 4.5 months.

To ensure complete experimental reproducibility and eliminate data leakage, our preprocessing pipeline (`src/preprocess.py`) implements a strict, multi-stage protocol:

1. **Recursive Frequency Filtering:** A single pass of frequency filtering can leave residual rare items if removing an item shortens a session below the minimum threshold. We therefore apply a *fixed-point recursive filter*: items appearing fewer than 5 times in the dataset are dropped, sessions shorter than 2 clicks are eliminated, and this pruning is repeated iteratively until both conditions hold simultaneously.
2. **Chronological Temporal Splitting:** Rather than random splitting (which causes severe future-data leakage in sequential tasks), we split sessions chronologically by their final timestamp. For Diginetica, the final 7 days form the test set; the 7 days immediately preceding the test window form the validation set; and all earlier sessions constitute the training set. For RetailRocket, the test and validation windows are each set to 14 days.
3. **Prefix-Target Decomposition:** Each processed session $S = [s_1, \ldots, s_l]$ is converted into $l - 1$ training examples: $([s_1], s_2), ([s_1, s_2], s_3), \ldots, ([s_1, \ldots, s_{l-1}], s_l)$.
4. **Contiguous One-Based Mapping:** Item IDs are mapped to contiguous integers in $1 \ldots |V|$. ID `0` is strictly reserved for batch padding.

Table 1: Statistics for the processed Diginetica benchmark.

Metric / Quantity	Value
Total unique candidate items ($\|V\|$)	42,068
Training prefix/target examples	679,395
Validation prefix/target examples	44,638
Test prefix/target examples	60,411
Temporal test window	7 days
Temporal validation window	7 days
Embedding matrix dimensions	$42,069 \times 100$
Total trainable parameters	4,358,300

Table 1 summarizes the resulting dataset statistics for the clean Diginetica benchmark.

3.3 Five-File Frozen Artifact Contract

To guarantee that different models (SR-GNN, SASRec, SSL variants) are evaluated on bit-for-bit identical data, our pipeline serializes each condition into an isolated directory containing exactly five files:

1. `train.txt`: Binary pickle of training pairs.
2. `validation.txt`: Binary pickle of validation pairs.
3. `test.txt`: Binary pickle of test pairs.
4. `item_mapping.pkl`: Raw item ID to 1-based index mapping.
5. `preprocessing_summary.json`: Metadata with record counts, split timestamps, and SHA-256 hashes of all files.

Every model-facing training script strictly validates these checksums before executing.

3.4 Controlled Noise Perturbation Protocols

To rigorously evaluate model robustness against real-world clickstream irregularities, we built an offline noise generator (`src/generate_noisy_test.py`). We define three parameterized perturbation operators acting on an observed prefix $S_{1:t} = [s_1, \ldots, s_t]$ with corruption probability $p \in \{0.05, 0.10, 0.20\}$:

- **Deletion Noise ($\mathcal{T}_{del}$):** Clicks within the prefix are randomly deleted with probability p. To preserve the minimal structural definition of a session graph, prefixes originally of length ≥ 2 are never reduced below length 2.
- **Substitution Noise ($\mathcal{T}_{sub}$):** Clicks within the prefix are randomly replaced with probability p by

an item sampled uniformly from V. The substituted item is constrained to differ from both the original item and the held-out target.

- **Mixed Noise ($\mathcal{T}_{mix}$):** Both deletion and substitution are applied sequentially at rate $p/2$.

Mathematical Target-Invariance Guarantee. In every noisy evaluation condition, the held-out target item s_{t+1} and the final transition into the target remain strictly invariant:

$$\text{target}(\mathcal{T}(S_{1:t})) \equiv s_{t+1}, \quad \forall \mathcal{T} \tag{3}$$

for all perturbation operators $\mathcal{T} \in \{\mathcal{T}_{del}, \mathcal{T}_{sub}, \mathcal{T}_{mix}\}$. This guarantees that all perturbations test the model's ability to recover intent from corrupted history without altering the evaluation ground truth.

4 Methodology: SR-GNN Architecture

4.1 Session Graph Construction

Given an observed prefix $S_{1:t} = [s_1, \ldots, s_t]$, we construct a directed graph $G_s = (V_s, E_s)$. The node set $V_s \subseteq V$ contains all unique items appearing in $S_{1:t}$. An edge $(v_i, v_j) \in E_s$ exists if item v_i is immediately followed by item v_j at least once in the prefix.

Following the official SR-GNN formulation, edges are binary (repeated transitions do not increase edge weight). The connection topology is represented by an adjacency matrix $A_s \in \mathbb{R}^{n \times 2n}$ (where $n = |V_s|$), formed by concatenating separately normalized incoming and outgoing transition matrices:

$$A_s = \left[A_s^{(in)} \parallel A_s^{(out)} \right] \tag{4}$$

where $A_{s,ij}^{(in)} = \frac{1}{d_i^{in}}$ if $(v_j, v_i) \in E_s$ and 0 otherwise, and $A_{s,ij}^{(out)} = \frac{1}{d_i^{out}}$ if $(v_i, v_j) \in E_s$ and 0 otherwise.

Because the same unique item can appear multiple times in a session, we define an alias mapping vector $\mathbf{p} = [p_1, \ldots, p_t] \in \{1, \ldots, n\}^t$ such that $s_k = V_s[p_k]$. This allows recovering the full temporal sequence after graph convolution.

4.2 Gated Graph Convolution

Every item $v \in V$ is mapped to an embedding vector $\mathbf{v} \in \mathbb{R}^d$ via an embedding lookup matrix $\mathbf{E} \in \mathbb{R}^{(|V|+1) \times d}$. Let $\mathbf{h}_i^{(0)} = \mathbf{v}_i$ denote the initial representation of node $i \in V_s$. At each propagation step $\tau \in \{1, \ldots, T\}$, nodes exchange messages through Gated Graph Neural Network (GGNN) steps (Li et al., 2015):

$$\mathbf{a}_i^{(\tau)} = \sum_{j=1}^{n} A_{s,ij}^{(in)} \mathbf{h}_j^{(\tau-1)} \mathbf{W}_{in} + \sum_{j=1}^{n} A_{s,ij}^{(out)} \mathbf{h}_j^{(\tau-1)} \mathbf{W}_{out} + \mathbf{b}$$

$$\mathbf{z}_i^{(\tau)} = \sigma\left(\mathbf{W}_z \mathbf{a}_i^{(\tau)} + \mathbf{U}_z \mathbf{h}_i^{(\tau-1)}\right), \quad \mathbf{r}_i^{(\tau)} = \sigma\left(\mathbf{W}_r \mathbf{a}_i^{(\tau)} + \mathbf{U}_r \mathbf{h}_i^{(\tau} \tag{5}$$

$$\widetilde{\mathbf{h}}_i^{(\tau)} = \tanh\left(\mathbf{W}_o \mathbf{a}_i^{(\tau)} + \mathbf{U}_o \left(\mathbf{r}_i^{(\tau)} \odot \mathbf{h}_i^{(\tau-1)}\right)\right) \tag{6}$$

$$\mathbf{h}_i^{(\tau)} = \left(1 - \mathbf{z}_i^{(\tau)}\right) \odot \mathbf{h}_i^{(\tau-1)} + \mathbf{z}_i^{(\tau)} \odot \widetilde{\mathbf{h}}_i^{(\tau)} \tag{7}$$

where $\mathbf{z}_i^{(\tau)}$ is the update gate, $\mathbf{r}_i^{(\tau)}$ is the reset gate, $\sigma(\cdot)$ is the sigmoid function, and $\odot$ is the Hadamard element-wise product. In our experiments, $T = 1$ propagation step is used, matching the baseline paper.

4.3 Hybrid Session Representation

After graph propagation, we extract the updated sequence representations using the alias vector: $\mathbf{x}_k = \mathbf{h}_{p_k}^{(T)}$ for $k \in \{1, \ldots, t\}$. SR-GNN constructs a hybrid session representation $\mathbf{s}_h$ by combining local and global preference:

Local Preference: The user's immediate, short-term intent is defined by the final clicked item: $\mathbf{s}_l = \mathbf{x}_t$.

Global Preference: The broader session context is aggregated using a soft-attention mechanism over all clicks in the sequence:

$$\alpha_k = \mathbf{q}^\top \sigma\left(\mathbf{W}_1 \mathbf{x}_t + \mathbf{W}_2 \mathbf{x}_k + \mathbf{c}\right), \quad \mathbf{s}_g = \sum_{k=1}^{t} \alpha_k \mathbf{x}_k \tag{8}$$

where $\mathbf{q} \in \mathbb{R}^d$, $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{d \times d}$, and $\mathbf{c} \in \mathbb{R}^d$.

Hybrid Session Embedding: The final session embedding $\mathbf{s}_h \in \mathbb{R}^d$ linearly projects the concatenation of local and global representations:

$$\mathbf{s}_h = \mathbf{W}_3 \left[\mathbf{s}_l \parallel \mathbf{s}_g\right], \quad \mathbf{W}_3 \in \mathbb{R}^{d \times 2d} \tag{9}$$

4.4 Prediction and Objective Function

The model computes recommendation candidate scores across all items in V via the dot product with their embedding vectors, normalized by softmax:

$$\hat{z}_i = \mathbf{s}_h^\top \mathbf{v}_i, \quad \hat{y}_i = \frac{\exp(\hat{z}_i)}{\sum_{j=1}^{|V|} \exp(\hat{z}_j)} \tag{10}$$

The model is trained end-to-end using the multi-class cross-entropy loss over the true one-hot next-item label $\mathbf{y}$:

$$\mathcal{L}_{rec} = -\sum_{i=1}^{|V|} y_i \log(\hat{y}_i) = -\log \hat{y}_{target} \quad (11)$$

5 Experimental Results and Analysis

5.1 Experimental Setup

Experiments were executed on an Apple M-series environment (Python 3.12, PyTorch 2.15). Training hyperparameters were strictly set to: embedding dimension $d = 100$, batch size $B = 100$, Adam optimizer with initial learning rate $\eta = 10^{-3}$, weight decay $\lambda = 10^{-5}$, and a step learning rate decay factor of 0.1 applied every 3 epochs. Training ran for a maximum of 30 epochs with early stopping patience of 10 epochs monitored on validation MRR@20.

Evaluation is performed using standard top-K ranking metrics ($K \in \{10, 20\}$):

- **Recall@K (P@K):** Proportion of test sessions where the true target item appears in the top-K recommendations.
- **MRR@K (Mean Reciprocal Rank):** Average of the reciprocal rank of the true target item if it appears within the top-K list, and 0 otherwise.

5.2 Baseline Reproduction on Diginetica

Training converged in 1,931 seconds (approximately 32 minutes on CPU). Validation MRR@20 peaked at **epoch 4** (Recall@20 = 50.69%, MRR@20 = 17.53%). Beyond epoch 4, training cross-entropy continued to drop smoothly while validation MRR declined, triggering early stopping at epoch 14. Restoring the best checkpoint (epoch 4) produced the test results in Table 2.

Table 2: Reproduction comparison on the Diginetica benchmark.

Model / Source	Recall@20	MRR@20
Published SR-GNN (Wu et al., 2019)	50.73%	17.59%
Our Reproduction (Seed 2028)	**49.55%**	**17.11%**
Absolute Difference	*-1.18%*	*-0.48%*

The small difference (-1.18% Recall@20) is directly attributable to our strict, leak-free chronological validation split (7 days preceding test), whereas the legacy protocol used a random 10% sample of sessions that included temporal overlap. This confirms that our PyTorch implementation faithfully reproduces the SR-GNN baseline.

Table 3: Performance of Clean Baseline and Model 2 across clean and noisy test conditions on Diginetica.

Test Condition	Clean Baseline R@20 / MRR@20	Model 2 (Aug.) R@20 / MRR@20
Clean Test Set	**49.55% / 17.11%**	49.48% / 17.03%
Deletion 5%	49.27% / 16.98%	49.24% / 16.94%
Deletion 10%	48.91% / 16.82%	48.88% / 16.79%
Deletion 20%	48.33% / 16.60%	48.29% / 16.55%
Mixed 5%	47.78% / 16.42%	47.71% / 16.36%
Mixed 10%	45.41% / 15.53%	45.33% / 15.48%
Mixed 20%	42.82% / 14.58%	42.75% / 14.52%
Substitution 5%	46.12% / 15.84%	46.04% / 15.78%
Substitution 10%	43.25% / 14.81%	43.19% / 14.75%
Substitution 20%	**40.75% / 13.88%**	40.68% / 13.82%

5.3 Robustness Evaluation under Synthetic Noise

We evaluated the trained clean baseline against test sets perturbed by deletion, substitution, and mixed noise at rates $p \in \{0.05, 0.10, 0.20\}$. In addition, to test whether noise injection during training can impart robustness, we trained an augmented baseline (**Model 2**) where 5% of training sessions were subjected to 10% deletion noise.

Table 3 presents the quantitative results across all conditions.

5.4 Key Findings and In-Depth Analysis

Our experimental results reveal three critical insights:

1. Asymmetric Vulnerability (Substitution vs. Deletion): Under 20% deletion noise, Recall@20 degrades by only 2.46% relative (from 49.55% to 48.33%). In stark contrast, 20% substitution noise causes a catastrophic 17.76% relative decline (down to 40.75%). When an item is deleted, the session graph loses intermediate connectivity, but the remaining items retain valid semantic associations. When an item is replaced by a random item, an entirely irrelevant node is injected into the graph, corrupting message passing and skewing the attention weights α_k in the global pooling mechanism.

2. Inadequacy of Naive Data Augmentation: Model 2 (trained with fixed 5% deletion noise) achieved 49.48% clean Recall@20 and 48.29% under 20% deletion. Naive deletion during training did not improve resilience against deletion noise and slightly impaired clean performance. This indicates that simple supervised noise injection is insufficient to regularize graph embeddings, provid-

ing clear empirical motivation for self-supervised contrastive learning.

3. Recency Sensitivity: Inspection of the attention weights in Eq. (9) shows that the local representation $\mathbf{s}_l = \mathbf{x}_t$ dominates candidate scoring. Any perturbation that touches the most recent item disproportionately degrades top-1 and top-5 ranking. This underscores the necessity of designing *recency-aware* contrastive augmentations.

6 Planned Self-Supervised Extension

6.1 Contrastive Session Learning

To make session graph embeddings resilient against clickstream corruption, we propose integrating an auxiliary Self-Supervised Learning (SSL) objective based on the InfoNCE contrastive loss (Oord et al., 2018).

Given a session prefix S, we generate two stochastic graph views $\widetilde{G}_s^{(1)}$ and $\widetilde{G}_s^{(2)}$ via stochastic graph perturbations:

$$\widetilde{G}_s^{(1)} \sim \mathcal{A}_{rec}(G_s), \quad \widetilde{G}_s^{(2)} \sim \mathcal{A}_{rec}(G_s) \tag{12}$$

where $\mathcal{A}_{rec}$ is our *recency-aware graph augmentation*. Passing both views through the shared SR-GNN encoder yields session representation vectors $\mathbf{z}_s^{(1)}, \mathbf{z}_s^{(2)} \in \mathbb{R}^d$.

For a mini-batch of B sessions, the two views of the same session serve as a positive pair $(\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)})$, while views of distinct sessions in the batch serve as negative pairs. The session-level contrastive loss is defined as:

$$\mathcal{L}_{ssl} = -\sum_{i=1}^{B} \log \frac{\exp\left(\frac{\text{sim}(\mathbf{z}_i^{(1)}, \mathbf{z}_i^{(2)})}{\tau}\right)}{\sum_{j=1}^{B} \exp\left(\frac{\text{sim}(\mathbf{z}_i^{(1)}, \mathbf{z}_j^{(2)})}{\tau}\right)} \tag{13}$$

where $\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$ denotes cosine similarity, and $\tau > 0$ is a temperature hyperparameter.

6.2 Recency-Aware Augmentation Operators

Standard graph contrastive learning applies uniform edge dropout or node masking. In session graphs, however, the final item s_t carries the overwhelming majority of predictive intent. Randomly dropping the last transition destroys the core recommendation signal.

We therefore enforce a *recency-preserving constraint*:

- The final observed item s_t and its incoming directed edge (s_{t-1}, s_t) are strictly protected and never masked or deleted.
- Historical transitions (s_i, s_{i+1}) for $i < t-1$ are subjected to stochastic edge dropout with rate $p_{edge} \in [0.1, 0.3]$.
- Subgraph node masking randomly masks historical nodes with rate $p_{node} \in [0.1, 0.2]$.

6.3 Joint Multi-Task Objective

The complete model will be trained end-to-end using a joint multi-task loss combining next-item cross-entropy and session contrastive regularization:

$$\mathcal{L}_{total} = \mathcal{L}_{rec} + \lambda \mathcal{L}_{ssl} \tag{14}$$

where $\lambda > 0$ balances the supervised and self-supervised objectives.

7 Risks, Challenges, and Mitigation

1. **Negative Sample Collisions in Mini-Batches:** In e-commerce clickstreams, two distinct sessions may exhibit nearly identical browsing intents (e.g., viewing two popular items in the same product category). Treating distinct sessions as hard negatives in InfoNCE can induce false-negative penalties.
   Mitigation: We will implement false-negative filtering based on session item overlap and experiment with decoupled contrastive learning.
2. **Hyperparameter Sensitivity (λ, τ):** Graph contrastive losses can collapse if temperature τ is poorly tuned, leading to gradient vanishing.
   Mitigation: We will perform grid search over $\tau \in \{0.05, 0.1, 0.2, 0.5\}$ and $\lambda \in \{0.01, 0.05, 0.1, 0.5\}$ on validation MRR@20.
3. **Computational Scalability:** Generating dual graph views per batch doubles the forward pass overhead.
   Mitigation: We have implemented batched graph collation where graph views share adjacency tensor operations.

8 Team Roles and Division of Responsibilities

The project responsibilities are divided across five technical modules:

- **Synthetic Perturbation Suite & Robustness Benchmark:** Designed and implemented the synthetic noise perturbation suite (`generate_noisy_test.py`); established the

mathematical target-invariance contract; executed the comprehensive 9-condition robustness benchmarks; and authored the empirical analysis.

- **Dataset Preprocessing Pipeline & Verification:** Designed the deterministic dataset preprocessing pipeline (`preprocess.py`); implemented recursive fixed-point frequency filtering; constructed the chronological train/validation/test temporal splits; and developed the five-file frozen artifact validation suite.
- **Graph Construction & Dynamic Collation:** Built the graph construction module (`session_graph.py`); implemented normalized directed adjacency tensor conversion, alias indexing for sequential state reconstruction, and the dynamic batch collator.
- **SR-GNN Architecture & Training Harness:** Implemented the vectorized SR-GNN model in modern PyTorch (`srgnn.py`); coded the Gated Graph Neural Network propagation layers, soft-attention hybrid session pooling, and the multi-class cross-entropy optimization harness.
- **Self-Supervised Formulation & Experiment Orchestration:** Formulated the upcoming Self-Supervised Learning architecture and recency-aware graph augmentation operators; set up multi-seed experiment tracking scripts; and conducted error distribution analysis.

9 Project Roadmap and Timeline

- **Weeks 1–2 (Completed):** Preprocessing pipeline, artifact verification, SR-GNN baseline reproduction on Diginetica (Seed 2028), and initial 9-condition noise robustness benchmark.
- **Weeks 3–4 (Nov 12–25):** Implement multi-seed baseline runs (5 seeds); implement recency-aware graph augmentations ($\mathcal{A}_{rec}$) and the InfoNCE objective; integrate RetailRocket benchmark.
- **Weeks 5–6 (Nov 26–Dec 5):** Conduct extensive ablation studies comparing clean SR-GNN, SASRec baseline, and SSL-enhanced SR-GNN across all noise conditions; evaluate Recall@10, Recall@20, MRR@10, and MRR@20.
- **Week 7 (Dec 6–12):** Final paper compilation, visualizations, code release preparation, and presentation delivery.

10 Conclusion

In this midterm status report, we established a reproducible foundation for session-graph recommendation research. We developed a deterministic data pipeline, fully reproduced the published SR-GNN baseline on Diginetica, and conducted an in-depth robustness evaluation across nine controlled corruption regimes. Our findings demonstrate that substitution noise induces severe degradation while simple supervised noise augmentation fails to provide robustness. Our upcoming work will focus on integrating recency-aware self-supervised contrastive learning to build robust, noise-resilient session recommendation models.

References

Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 639–648.

Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based recommendations with recurrent neural networks. In *International Conference on Learning Representations (ICLR)*.

Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *IEEE International Conference on Data Mining (ICDM)*, pages 197–206.

Thomas N Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*.

Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. 2015. Gated graph sequence neural networks. *arXiv preprint arXiv:1511.05493*.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*.

Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2010. Factorizing personalized markov chains for next-basket recommendation. In *Proceedings of the 19th International Conference on World Wide Web*, pages 811–820.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph attention networks. In *International Conference on Learning Representations (ICLR)*.

Shu Wu, Yuyuan Tang, Yanmin Zhu, Liang Wang, Xing Xie, and Tieniu Tan. 2019. Session-based recommendation with graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 346–353.

Xin Xia, Hongzhi Yin, Junliang Yu, Qinyong Wang, Lizhen Cui, and Xiangliang Zheng. 2021. Self-supervised hypergraph convolutional networks for session-based recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 4503–4511.

Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Wen Ji-Rong. 2020. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. pages 1893–1902.